Quiz 6

- Can Prim's Algorithm work on Negative weight Cycle graphs?

Quiz 6 cy-b

- Is it possible for Prim's algorithm to construct a Minimum Spanning Tree (MST) that excludes the edge with the smallest weight in the graph?

National University of Computer & Emerging Sciences

Hashing

What is a database?

- Structured collection of data.

Student ID	First Name	Last Name	Email	Major	Faculty
200120	Kate	West	kwest@email.com	Music	Arts
200121	Julie	McLain	jmclain@email.com	Finance	Business
200122	Tom	Erlich	terlich@email.com	Sculpture	Arts
200123	Mark	Smith	msmith@email.com	Biology	Science
200124	Jen	Foster	jfoster@email.com	Physics	Science
200125	Matt	Knight	mknight@email.com	Finance	Business
200126	Karen	Weaver	kweaver@email.com	Music	Arts
200127	John	Smith	jsmith@email.com	Sculpture	Arts
200128	Allison	Page	apage@email.com	History	Humanities
200129	Craig	Cambell	ccambell@email.com	Music	Arts
200130	Steve	Edwards	sedwards@email.com	Biology	Science
200131	Mike	Williams	mwilliams@email.com	Linguistics	Humanities
200132	Jane	Reid	jreid@email.com	Music	Arts

The Dictionary ADT

- a dictionary (table) is an abstract model of a database
- a dictionary stores key-element pairs
- the main operation supported by a dictionary is searching by key

Examples

- Telephone directory
- Library catalogue
- Books in print: key ISBN

- Any other example?

Applications

- Keeping track of customer account information at a bank
  - Search through records to check balances and perform transactions
- Keep track of reservations on flights
  - Search to find empty seats, cancel/modify reservations
- Search engine
  - Looks for all documents containing a given word

Main Issues

- Size
- Operations: search, insert, delete
- What will be stored in the dictionary?
- How will items be identified?

The Dictionary ADT

- simple container methods:
  - size()
  - isEmpty()
  - elements()
- query methods:
  - findElement(k)
  - findAllElements(k)

The Dictionary ADT

- update methods:
  - insertItem(k, e)
  - removeElement(k)
  - removeAllElements(k)

A case scenario

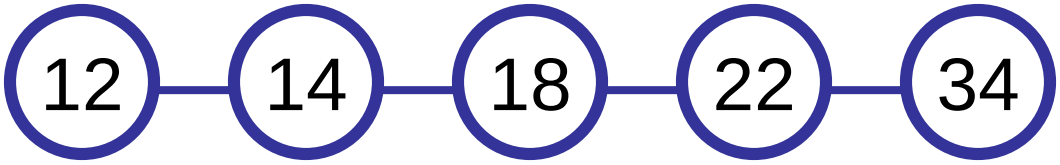
- ATCL is a large phone company, and they want to provide enhanced caller ID capability:
  - given a phone number, return the caller's name
  - phone numbers are in the range 0 to $R = 10^{10}-1$
  - n is the number of phone numbers used
  - want to do this as efficiently as possible

Implementing a Dictionary

- ***unordered sequence***
  - searching and removing takes n steps
  - inserting takes only constant time, one step
  - applications to log files (frequent insertions, rare searches and removals) 34 14 12 22 18

Implementing a Dictionary

- ***array-based ordered sequence***
  (assumes keys can be ordered)
  - searching can be efficient
  - inserting and removing takes n steps
  - application to look-up tables
  (frequent searches, rare insertions and removals)



A case scenario

- We know two ways to design this dictionary:
- ***A balanced search tree*** (AVL, b trees) --- good space usage and search time, but can we reduce the search time to constant?

- Direct Addressing

Direct Addressing

- Assumptions:
  - Key values are distinct
  - Each key is drawn from a universe U = {0, 1, . . . , m - 1}
- Idea:
  - Store the items in an array, indexed by keys

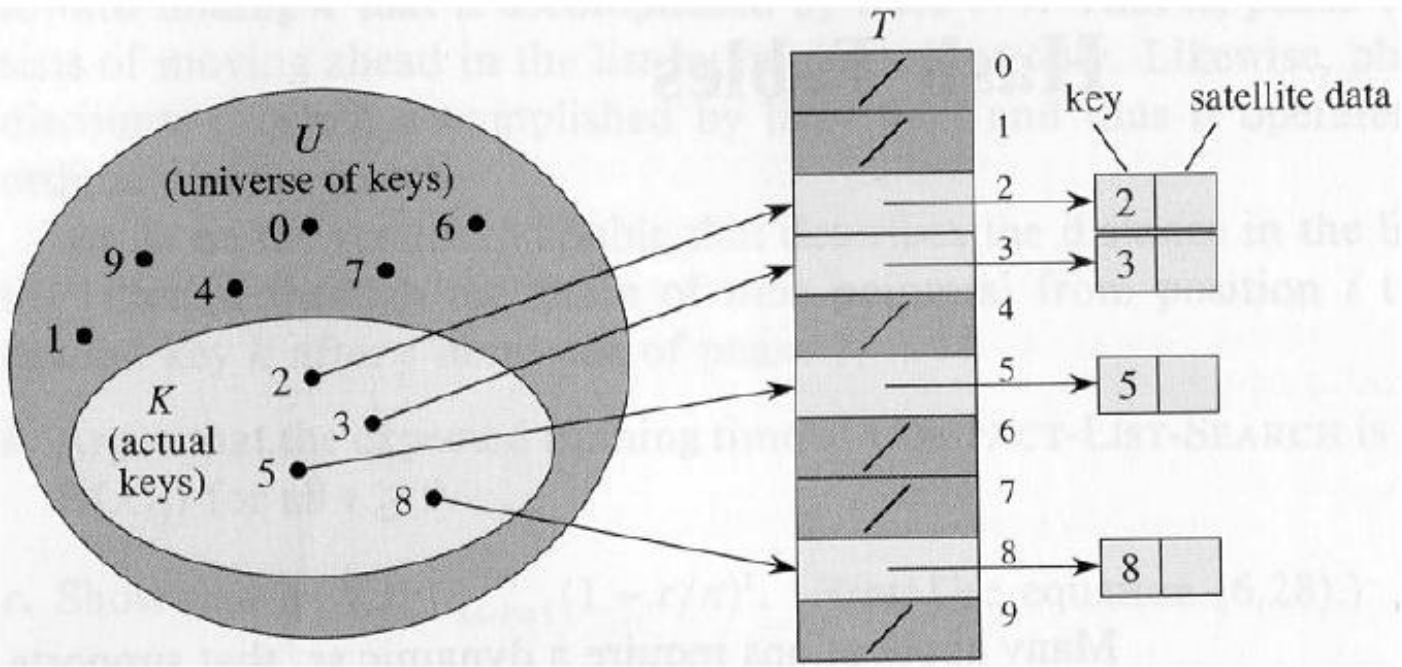
- **Direct-address table** representation:
  - An array T[0 . . . m - 1]
  - Each **slot**, or position, in T corresponds to a key in U
  - For an element x with key k, a pointer to x (or x itself) will be placed in location T[k]
  - If there are no elements with key k in the set, T[k] is empty, represented by NIL

Direct Addressing

- Each cell is thought of as a bucket or a container
  - Holds key element pairs
  - In array A of size N, an element *e* with key *k* is inserted in *A[k]*.

(null)	(null)	…	Hammad	…	(null)
000-000-0000	000-000-0001		401-863-7639 ...		999-999-9999

Direct Addressing (cont'd)



(insert/delete in O(1) time)

Operations

Alg.: DIRECT-ADDRESS-SEARCH(T, k)

return T[k]

Alg.: DIRECT-ADDRESS-INSERT(T, x)

T[key[x]] ← x

Alg.: DIRECT-ADDRESS-DELETE(T, x)

T[key[x]] ← NIL

- Running time for these operations: $O(1)$

Advantages with Direct Addressing

- Direct Addressing is the most efficient way to access the data since
  - It takes only single step for any operation on direct address table.
  - It works well when the Universe U of keys is reasonable small.

Difficulty with Direct Addressing

When the universe *U* is very large…

- Storing a table *T* of size *U* may be impractical, given the memory available on a typical computer.
- The set *K* of the keys actually stored may be so small relative to *U* that most of the space allocated for *T* would be wasted.

An Example

- A table, 50 students in a class.
- The key, 9 digit *SSN*, used to identify each student.
- Number of different 9 digit number=10^9
- The fraction of actual keys needed. $50/10^9$, **<u>0.000005%</u>**
- Percent of the memory allocated for table wasted, **<u>99.999995%</u>**

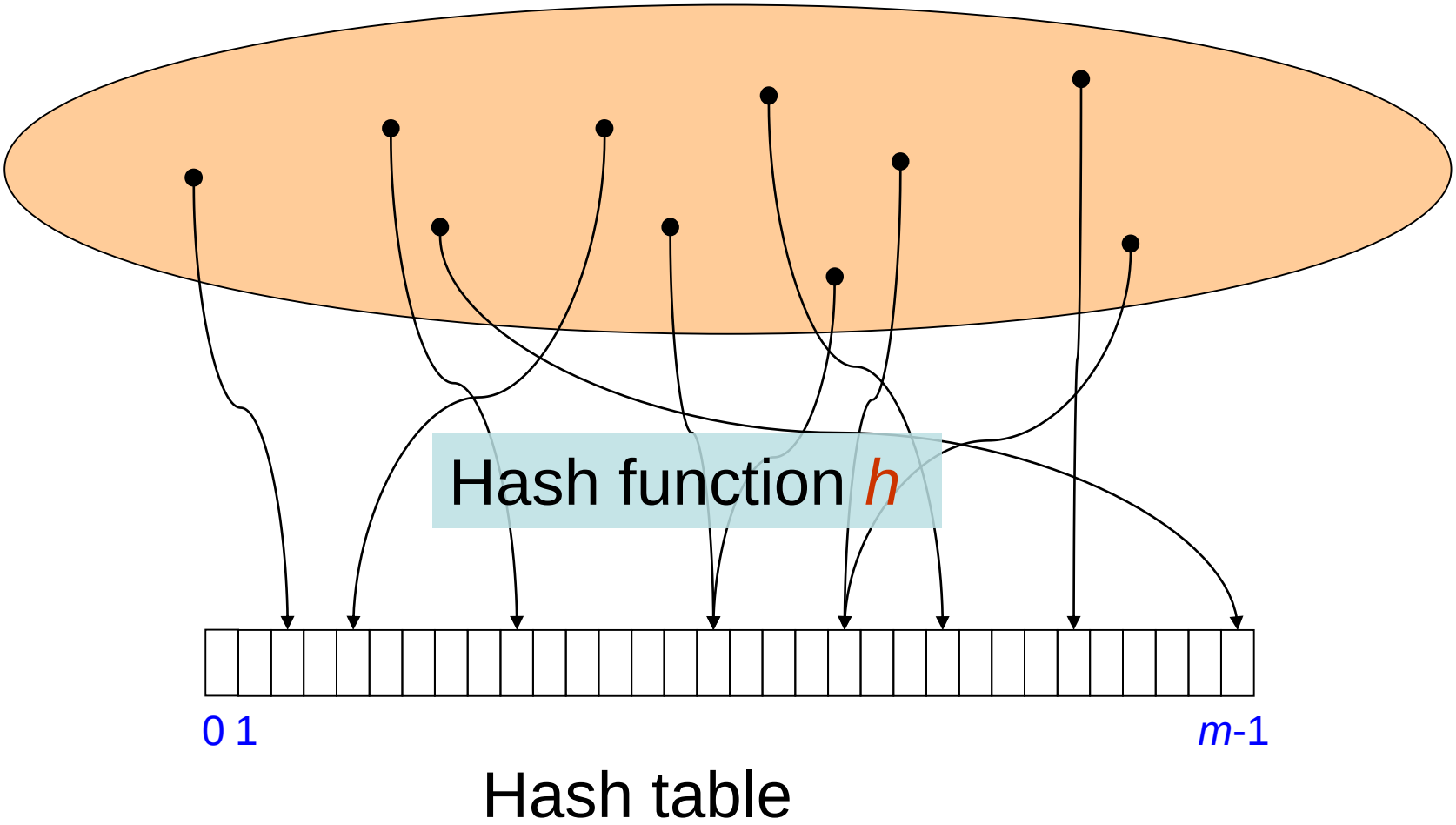
Hashing

- Suppose we were to come up with a “magic function” that, given a value to search for, would tell us exactly where in the array to look
  - If it’s in that location, it’s in the array
  - If it’s not in that location, it’s not in the array

- This function would have no other purpose
- This function is called a hash function because it “makes hash” of its inputs

Hashing

Huge universe *U*



Example (ideal) hash function

- Suppose our hash function gave us the following values:

  hashCode("apple") = 5
  hashCode("watermelon") = 3
  hashCode("grapes") = 8
  hashCode("cantaloupe") = 7
  hashCode("kiwi") = 0
  hashCode("strawberry") = 9
  hashCode("mango") = 6
  hashCode("banana") = 2

0	kiwi
1	
2	banana
3	watermelon
4	
5	apple
6	mango
7	cantaloupe
8	grapes
9	strawberry

Finding the hash function

- How can we come up with this magic function?
- In general, we cannot--there is no such magic function ☹
  - In a few specific cases, where all the possible values are known in advance, it has been possible to compute a perfect hash function
- What is the next best thing?
  - A perfect hash function would tell us exactly where to look
  - In general, the best we can do is a function that tells us where to *start* looking!

Example imperfect hash function

- Suppose our hash function gave us the following values:
  - hash("apple") = 5
    hash("watermelon") = 3
    hash("grapes") = 8
    hash("cantaloupe") = 7
    hash("kiwi") = 0
    hash("strawberry") = 9
    hash("mango") = 6
    hash("banana") = 2
    hash("honeydew") = 6
  - Now what?

0	kiwi
1	
2	banana
3	watermelon
4	
5	apple
6	mango
7	cantaloupe
8	grapes
9	strawberry

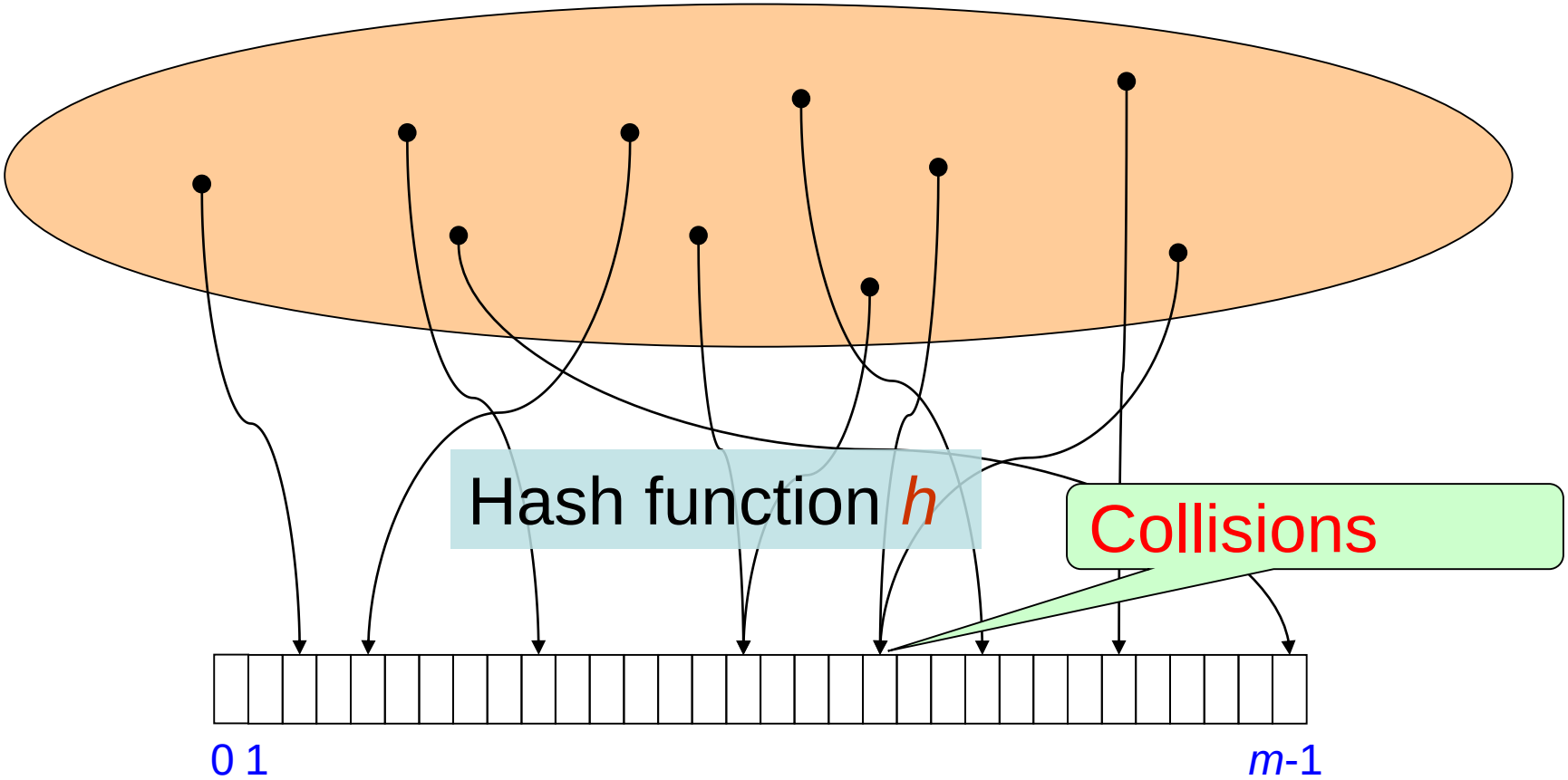
Collisions

- When two values hash to the same array location, this is called a collision

- Collisions are normally treated as “first come, first served”—the first value that hashes to the location gets it
- We have to find something to do with the second and subsequent values that hash to this same location

Hashing

Huge universe *U*



Hash table

Handling collisions

- What can we do when two different values attempt to occupy the same place in an array?
  - **Solution #1:** Search from there for an empty location
    - Can stop searching when we find the value *or* an empty location
    - Search must be end-around
  - **Solution #2:** Use a second hash function
    - ...and a third, and a fourth, and a fifth, ...
  - **Solution #3:** Use the array location as the header of a linked list of values that hash to this location
- All these solutions work, provided:
  - We use the same technique to *add* things to the array as we use to *search* for things in the array

Insertion, I

- Suppose you want to add seagull to this hash table
- Also suppose:
  - hashCode(seagull) = 143
  - table[143] is not empty
  - table[143] != seagull
  - table[144] is not empty
  - table[144] != seagull
  - table[145] is empty
- Therefore, put seagull at location 145

. . .	
141	
142	robin
143	sparrow
144	hawk
145	seagull
146	
147	bluejay
148	owl
. . .	

Searching, I

- Suppose you want to look up seagull in this hash table
- Also suppose:
  - hashCode(seagull) = 143
  - table[143] is not empty
  - table[143] != seagull
  - table[144] is not empty
  - table[144] != seagull
  - table[145] is not empty
  - table[145] == seagull !
- We found seagull at location 145

. . .	
141	
142	robin
143	sparrow
144	hawk
145	seagull
146	
147	bluejay
148	owl
. . .	

Searching, II

- Suppose you want to look up `cow` in this hash table
- Also suppose:
  - `hashCode(cow) = 144`
  - `table[144]` is not empty
  - `table[144] != cow`
  - `table[145]` is not empty
  - `table[145] != cow`
  - `table[146]` is empty
- If `cow` were in the table, we should have found it by now
- Therefore, it isn't here

. . .	
141	
142	robin
143	sparrow
144	hawk
145	seagull
146	
147	bluejay
148	owl
. . .	

Insertion, II

- Suppose you want to add hawk to this hash table
- Also suppose
  - hashCode(hawk) = 143
  - table[143] is not empty
  - table[143] != hawk
  - table[144] is not empty
  - table[144] == hawk
- hawk is already in the table, so do nothing

. . .	
141	
142	robin
143	sparrow
144	hawk
145	seagull
146	
147	bluejay
148	owl
. . .	

Insertion, III

- Suppose:
  - You want to add cardinal to this hash table
  - hashCode(cardinal) = 147
  - The last location is 148
  - 147 and 148 are occupied
- Solution:
  - Treat the table as circular; after 148 comes 0
  - Hence, cardinal goes in location 0 (or 1, or 2, or ...)

. . .	
141	
142	robin
143	sparrow
144	hawk
145	seagull
146	
147	bluejay
148	owl

Clustering

- One problem with the above technique is the tendency to form "clusters"
- A cluster is a group of items not containing any open slots
- The bigger a cluster gets, the more likely it is that new values will hash into the cluster, and make it ever bigger
- Clusters cause efficiency to degrade
- Here is a *non*-solution: instead of stepping one ahead, step n locations ahead
  - The clusters are still there, they're just harder to see
  - some table locations may be never checked

Solution #2: Rehashing

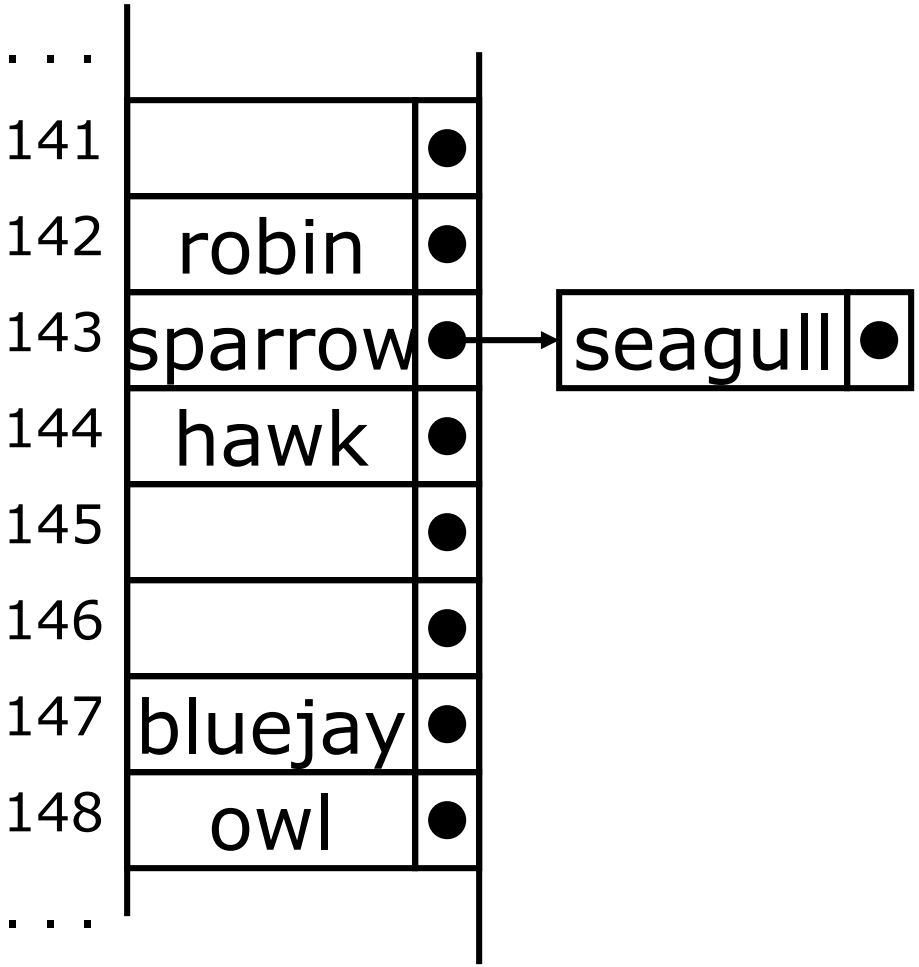
- In the event of a collision, another approach is to rehash: compute another hash function
  - Since we may need to rehash many times, we need an easily computable sequence of functions

Solution #2: Rehashing

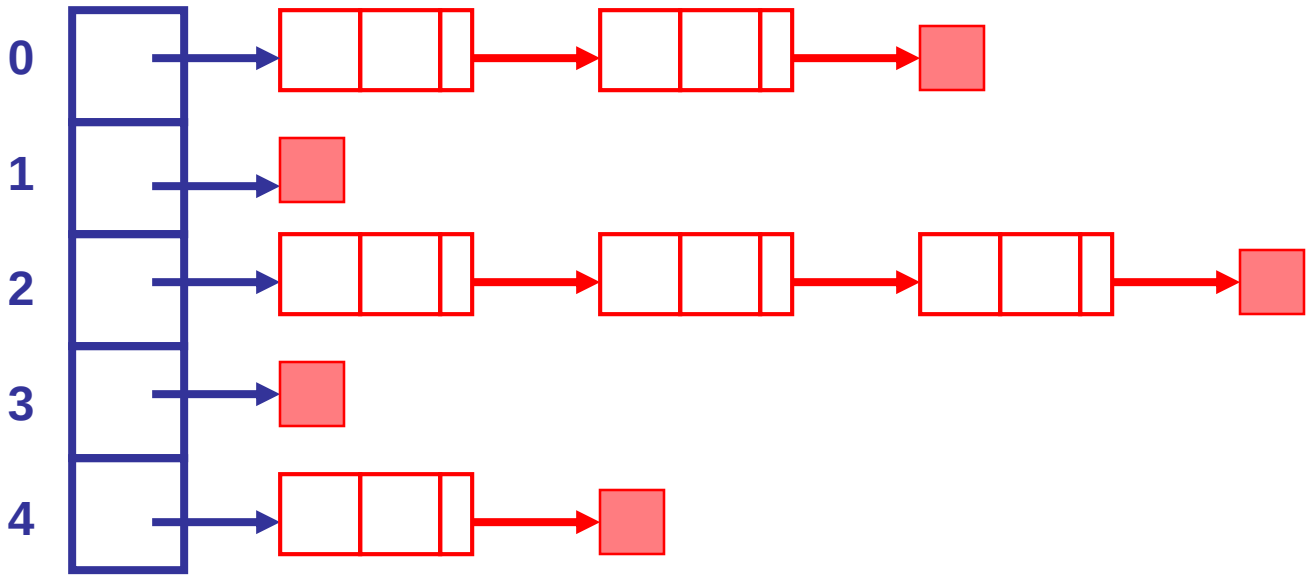
- Simple example: in the case of hashing Strings, we might take the previous hash code and add the length of the String to it
  - Probably better if the length of the string was not a component in computing the original hash function
- Possibly better yet: add the length of the String plus the number of probes made so far
  - Problem: are we sure we will look at every location in the array?
- Rehashing is a fairly uncommon approach, and we won't pursue it any further here

Solution #3: Chaining

- The previous solutions used open address hashing: all entries went into a "flat" (unstructured) array
- Another solution is to make each array location the header of a linked list of values that hash to that location

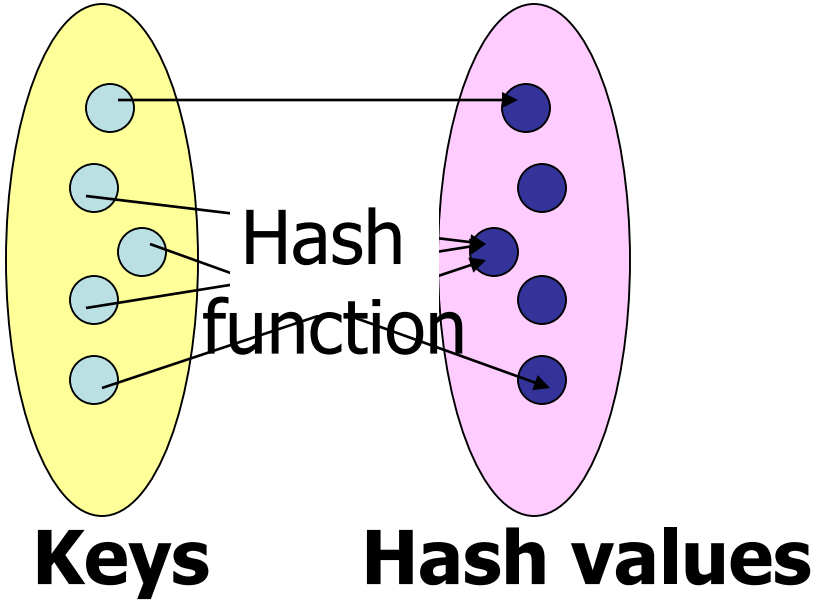


Chaining



What is a Hash function?

A hash function is a **mapping** between a set of input values (**Keys**) and a set of integers, known as **hash values**.



Choosing a good hash-function is crucial !!

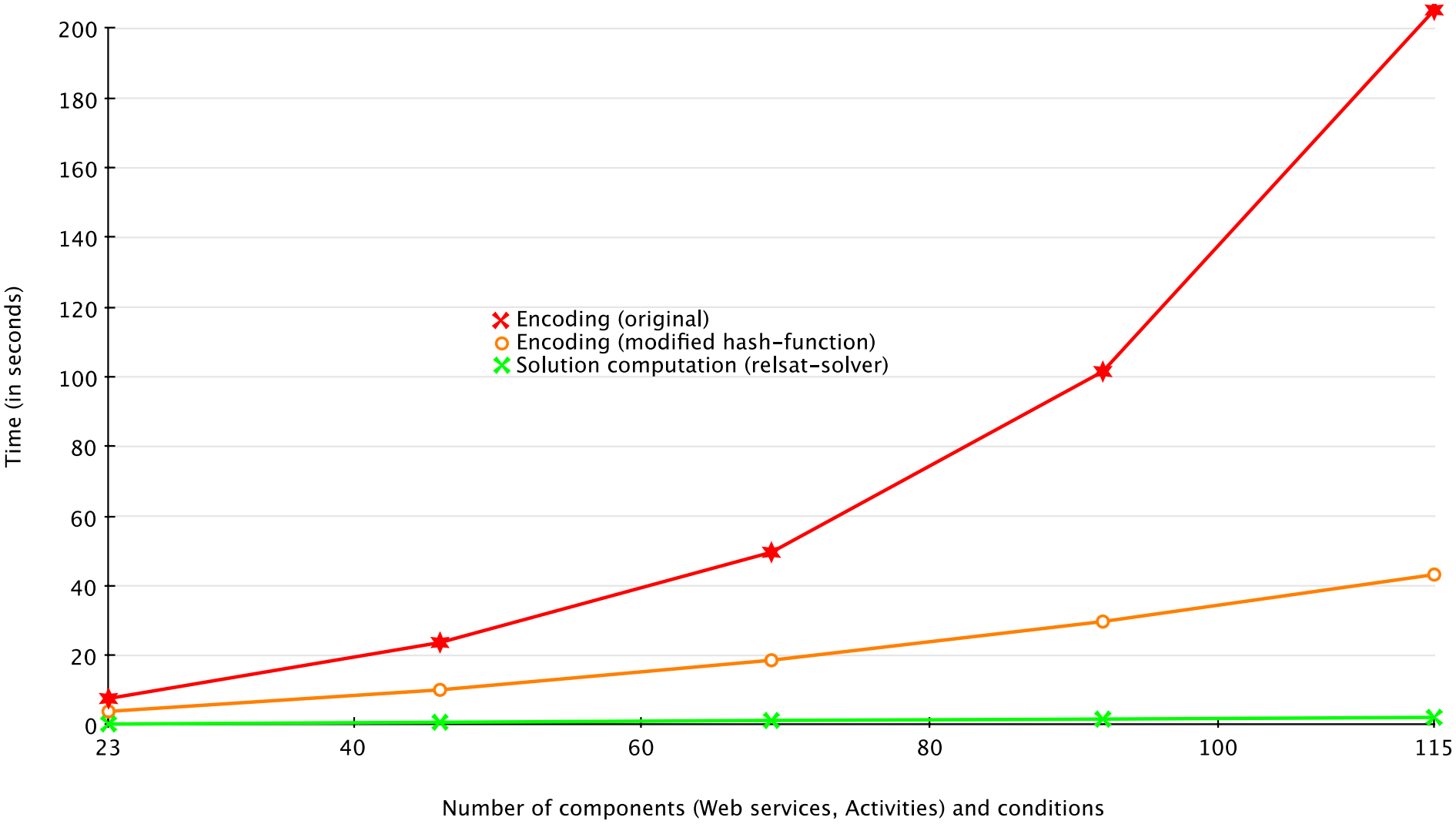
A case-study

- **Discrete Event Calculus Reasoner** is a tool developed by IBM that is being used to solve problems in different domains.
- The tool does not scale well, with increase in input size.
- Careful analysis revealed that the hash-function being used is not distributing the keys uniformly resulting in lot of collisions.

A case-study

- The hash function was being calculated on first six-characters of the input string
- However, our analysis revealed that the first six characters normally stay the same, only last six-characters differ.
- What does it mean? How to change it?

A case-study


200
180
160
140
120
100
80
60
40
20
0
Time (in seconds)
Encoding (original)
Encoding (modified hash-function)
Solution computation (relsat-solver)
23
40
60
80
100
115
Number of components (Web services, Activities) and conditions

The properties of a good hash function

- Rule1: The hash value is fully determined by the data being hashed.
- Rule2: The hash function uses all the input data.
- Rule3: The hash function uniformly distributes the data across the entire set of possible hash values.
- Rule4: The hash function generates very different hash values for similar strings.

An example of a hash function

```
int hash(char *str, int table_size)
{
    int sum=0;
   //sum up all the characters in the string
    for(;*str; str++) sum+=*str
   //return sum mod table_size
  return sum%table_size;
}
```

Analysis of example

- **Rule1: Satisfies**, the hash value is fully determined by the data being hashed, the hash value is just the sum of all input characters.

- **Rule2: Satisfies**, Every character is summed.

Analysis of example (contd.)

- **Rule3: Breaks**, from looking at it, it is not obvious that it doesn't uniformly distribute the strings, but if you were to analyze this function for larger input string, you will see certain statistical properties which are bad for a hash function.

- **Rule4: Breaks**, hash the string "CAT", now hash the string "ACT", they are the same, a slight variation in the string should result in different hash values, but with this function often they don't.

Methods to create hash functions

- **Division method**
- **Multiplication method**

The Division Method

- **Idea:**
  - Map a key k into one of the m slots by taking the remainder of k divided by m

$$h(k) = k \bmod m$$

- **Advantage**:
  - fast, requires only one operation
- **Disadvantage**:
  - Certain values of m are bad, e.g.,
    - power of 2
    - non-prime numbers

Restrictions on value of *m*

M should not be a power of 2, since if $m=2^p$ then h(k) is just the p lowest order bits of k.

Key	Binary	K mod 8
8	1000	0
7	111	7
12	1100	4
34	100010	2
56	111000	0
78	1001110	6
90	1011010	2
23	10111	7
45	101101	5
67	1000011	3

Example - The Division Method

- Choose m to be a prime, not close to a power of 2

A useful sequence of primes for this purpose is:

251, 509, 1021, 2039, 4093, 8191, 16381, 32749, 65521,

Multiplication method

- Using a random real number *f* in the range *[0,1).*
- The fractional part of the product *f*key* yields a number in the range *0 to 1*.
- When this number is multiplied by m (hash table size), the integer portion of the product gives the hash value in the range *0 to m-1*

More on multiplication method

- Choose $m = 2^P$
- For a constant A, $0 < A < 1$:
- $h(k) = \lfloor m\,(kA - \lfloor kA \rfloor) \rfloor$
- Value of A should not be close to 0 or 1
- *Knuth* says good value of A is 0.618033
- If $k=123456$, $m=10000$, and A as above

$$h(k)= \lfloor 10000*(123456*A- \lfloor 123456*A \rfloor)$$
$$= \lfloor 10000*0.0041151 \rfloor$$
$$=41$$

Efficiency

- Hash tables are actually surprisingly efficient
- Until the table is about 70% full, the number of probes (places looked at in the table) is typically only 2 or 3
- Sophisticated mathematical analysis is required to *prove* that the expected cost of inserting into a hash table, or looking something up in the hash table, is O(1)
- Even if the table is nearly full (leading to long searches), efficiency is usually still quite high

References (Homework)

Map STL

- http://www.cprogramming.com/tutorial/stl/stlmap.html

Hash Function in C++ (GCC)

- http://www.cplusplus.com/reference/functional/hash/
- http://stackoverflow.com/questions/19411742/what-is-the-default-hash-function-used-in-c-stdunordered-map
  - http://gcc.gnu.org/git/?p=gcc.git;a=blob_plain;f=libstdc%2b%2b-v3/libsupc%2b%2b/hash_bytes.cc;hb=HEAD

Finals are here !

- Practice all algorithms studied during the course (at least twice)
- Some practice problems:
  - http://www.spoj.com/problems/classical/
  - https://leetcode.com/problemset/algorithms/
  - http://www.codechef.com/problems/easy